Architectural Blueprint and Implementation Strategy for Eco-Neuro Sentinel: A Privacy-First Neuro-Symbolic System

Executive Analysis

The digital health landscape is undergoing a tectonic shift, moving from passive data repositories to active, neuro-symbolic reasoning engines that synthesize heterogeneous data streams into actionable medical intelligence. The 'Eco-Neuro Sentinel' project represents a vanguard implementation of this paradigm, designed to operate at the intersection of individual biological metrics and macroscopic environmental telemetry. The core objective is to correlate real-time respiratory symptoms with hyper-local air quality indices in Bucharest, utilizing a privacy-first architecture that strictly adheres to the General Data Protection Regulation (GDPR).

This report outlines a comprehensive, exhaustive development plan for the Eco-Neuro Sentinel. It leverages the synergistic capabilities of the Supabase ecosystem—specifically PostgreSQL, PostGIS, pgvector, and Edge Functions—and the OpenAI computational stack. The architecture is "neuro-symbolic" in that it fuses the probabilistic reasoning of Large Language Models (LLMs) with the deterministic, rule-based logic of SQL constraints and spatial geometry.[1]

The following analysis is structured into five distinct architectural phases. Each phase provides a deep theoretical underpinning, analyzing the "why" and "how" of the technology choices, followed by a high-fidelity 'Developer Prompt'. These prompts are engineered to be ingested by AI coding assistants, containing all necessary context, dependency requirements, and logical constraints to generate production-grade code.

Phase I: The Data Persistence Layer (Supabase SQL & RLS)

1.1 Architectural Theory: The Convergence of Relational and Vector Models

In traditional health informatics, data persistence is often bifurcated: structured patient logs reside in relational databases (RDBMS), while unstructured knowledge bases (such as medical

journals or regulatory directives) are indexed in separate search engines like Elasticsearch. Eco-Neuro Sentinel collapses this dichotomy by utilizing PostgreSQL as a unified persistence layer. This consolidation is made possible by the pgvector extension, which transforms Postgres into a vector database capable of storing and querying high-dimensional embeddings alongside standard scalar data types.[3]

The necessity for this unified approach stems from the complex nature of the query patterns required by the application. To answer a question like "Is the patient's asthma flare-up in Sector 1 correlated with a violation of the EU Air Quality Directive?", the database must simultaneously perform:

1. **Spatial Filtering:** A geometric containment query to locate the patient within a specific Bucharest sector (PostGIS).
2. **Temporal Filtering:** A time-range query to match the symptom onset with pollution spikes (Standard SQL).
3. **Semantic Retrieval:** A vector similarity search to find relevant clauses in the regulatory text that define "hazardous thresholds" for specific pollutants (pgvector).[2]

1.2 High-Dimensional Indexing: The HNSW Algorithm

While pgvector allows for the storage of vectors (in this case, 1536-dimensional vectors generated by OpenAI's text-embedding-3-small), the efficiency of retrieval is paramount. A sequential scan (checking every vector in the database) creates a latency penalty that scales linearly with dataset size—unacceptable for a real-time triage app.

To mitigate this, we employ the Hierarchical Navigable Small World (HNSW) indexing algorithm. HNSW is superior to the older Inverted File Flat (IVFFlat) method because it is graph-based rather than cluster-based. It constructs a multi-layered proximity graph where each node (vector) is linked to its nearest neighbors. The search process mimics a "small world" network traversal: it begins at the top layer with long-range links (skipping over vast regions of the vector space) and progressively drills down to lower layers with short-range, dense connections. This logarithmic time complexity ($O(\log N)$) ensures that retrieval times remain millisecond-fast even as the knowledge base grows to millions of vectors.[3]

Furthermore, recent optimizations in pgvector (v0.6.0+) introduce parallel index builds. This allows the database to utilize multiple CPU cores during the computationally intensive graph construction phase, reducing build times by up to 30x. This is critical for the maintenance lifecycle of the Eco-Neuro Sentinel, allowing for rapid re-indexing when new regulatory documents are ingested.[4]

1.3 Geospatial Calculus with PostGIS

The application's reliance on location data necessitates the postgis extension, the industry standard for spatial SQL. Storing latitude and longitude as simple float columns is insufficient for complex spatial topology. PostGIS introduces the GEOGRAPHY data type, which calculates

distances and areas on the geodetic spheroid (the curved surface of the Earth) rather than a flat Cartesian plane. This distinction is vital for accurate sector mapping in a city like Bucharest.

We utilize the Generalized Search Tree (GiST) index for spatial columns. Unlike B-Trees, which order data in a single dimension, GiST indexes use bounding box hierarchies (R-Trees) to quickly eliminate geometries that do not intersect with the query window. This enables the backend to execute Point-in-Polygon queries—determining which administrative sector a user is currently standing in—with negligible computational overhead.[2]

1.4 Privacy by Design: Row Level Security (RLS)

In the context of the EU's GDPR, the protection of health data (Special Category Data) is non-negotiable. Supabase exposes PostgreSQL's native Row Level Security (RLS), a primitive that enforces access control at the database row level. This is a significant architectural advantage over application-level logic. If a security flaw exists in the API middleware or the frontend code, the database itself remains secure because the RLS policy acts as a final, immutable firewall.

For Eco-Neuro Sentinel, we define a "Zero Trust" policy for the patient_logs table. The RLS policy is cryptographically bound to the user's authentication token (JWT). The database kernel evaluates the auth.uid() function against the user_id column for every SELECT, UPDATE, or DELETE operation. If the IDs do not match, the query returns an empty set, rendering the data invisible to the requester. This ensures that even if an attacker manages to execute a raw SQL query via the API, they can only access data belonging to the compromised account, preventing mass data exfiltration.[2]

1.5 Developer Prompt 1: The Foundation

The following prompt provides the blueprint for the database schema. It commands the enabling of extensions, the definition of strict data types, and the application of granular RLS policies.

Developer Prompt 1: The Foundation (Supabase SQL & RLS)

Context:
You are a Senior Database Architect designing the persistence layer for 'Eco-Neuro Sentinel', a GDPR-compliant health monitoring application. The system requires a convergence of geospatial tracking, vector-based semantic search, and strictly isolated patient data.
Goal:
Write a comprehensive, executable SQL script to initialize the Supabase project. This script

must enable necessary extensions, define the schema with precise data types, and implement a "Privacy by Design" security model using Row Level Security (RLS).

Requirements:

1. **Extensions:**
   - pgvector: Enable for storing 1536-dimensional embeddings from OpenAI.[3]
   - postgis: Enable for geospatial calculations (Point-in-Polygon).[2]
2. **Schema Definition:**
   - **profiles Table:**
     - id: UUID (Primary Key, references auth.users on delete cascade).
     - full_name: TEXT.
     - sector: TEXT (e.g., 'Sector 1').
     - created_at: TIMESTAMPTZ (default now()).
   - **patient_logs Table (High Sensitivity):**
     - id: UUID (Primary Key, default gen_random_uuid()).
     - user_id: UUID (Foreign Key to profiles.id, NOT NULL).
     - symptom_text: TEXT (The raw description of symptoms).
     - severity_score: INTEGER (Check constraint: 1-10).
     - log_location: GEOGRAPHY(Point, 4326) (Stores precise WGS84 coordinates).
     - created_at: TIMESTAMPTZ.
   - **air_quality Table (Public Utility):**
     - id: BIGINT (Identity).
     - sector_name: TEXT.
     - pm25: DOUBLE PRECISION.
     - pm10: DOUBLE PRECISION.
     - boundary: GEOGRAPHY(Polygon, 4326) (Defines the sector's shape).
     - last_updated: TIMESTAMPTZ.
   - **knowledge_base Table (RAG Source):**
     - id: BIGINT (Identity).
     - content_chunk: TEXT.
     - embedding: VECTOR(1536).
     - metadata: JSONB (Stores source page, article number).
3. **Indexing Strategy:**
   - **Vector Index:** Create an HNSW index on knowledge_base.embedding using vector_cosine_ops. Configure parameters m=16 and ef_construction=64 to balance build time with search recall.[3]
   - **Spatial Index:** Create GiST indexes on patient_logs.log_location and air_quality.boundary to optimize ST_Intersects and ST_Contains queries.
4. **Row Level Security (RLS) Policies:**
   - Enable RLS on all tables (ALTER TABLE... ENABLE ROW LEVEL SECURITY).
   - **profiles**: Users can SELECT and UPDATE their own rows (auth.uid() = id).
   - **patient_logs**: STRICT ISOLATION. Users can SELECT and INSERT only their own rows. No public access.

- **air_quality**: Public SELECT for authenticated users. INSERT/UPDATE restricted to service_role only.
- **knowledge_base**: Public SELECT for authenticated users. INSERT/UPDATE restricted to service_role.

5. **RPC Function for Similarity Search:**
   - Define a PL/pgSQL function match_documents that accepts query_embedding vector(1536), match_threshold float, and match_count int.
   - The function must return chunks with a cosine similarity (1 - (embedding <=> query_embedding)) greater than the threshold, ordered by similarity.

Output:
Provide the complete SQL code block, heavily commented to explain the security implications of each policy.

Phase II: The Knowledge Ingestion Pipeline (Python RAG Script)

2.1 Architectural Theory: From Unstructured Text to Semantic Vectors

The "Knowledge Ingestion" phase is the bridge between static regulatory documents and dynamic AI reasoning. The EU Air Quality Directive is a dense legal text containing crucial thresholds (e.g., "PM2.5 must not exceed 25 µg/m^3 annually"). For an AI agent to use this information effectively, the text must be transformed into a format that supports semantic retrieval—specifically, vector embeddings.

The process involves a sophisticated Extract-Transform-Load (ETL) pipeline. The "Extract" phase utilizes PDF parsing libraries to liberate text from the binary format. However, raw extraction often results in "noise"—headers, footers, and page numbers that interrupt the flow of sentences. The pipeline must sanitize this stream before processing.

2.2 Semantic Chunking Strategy

The "Transform" phase is where the complexity lies. Feeding an entire 200-page directive into an embedding model is ineffective; the resulting vector would be a diluted average of the entire document, lacking specific semantic resolution. Conversely, splitting the text by character count (e.g., every 500 characters) risks severing a regulation from its sub-clauses.

We employ **Recursive Character Text Splitting**. This algorithm attempts to split text at natural semantic boundaries (paragraphs), then sentences, then words. By setting a chunk_size of roughly 1000 characters with a chunk_overlap of 200 characters, we ensure that the "connective tissue" of the text is preserved. If a sentence describing a penalty for pollution violation spans across a split, the overlap ensures the context is present in both

resulting chunks, preventing information loss during retrieval.[6]

2.3 The OpenAI Embedding Model

For the embedding layer, we utilize text-embedding-3-small. This model represents a significant efficiency leap over its predecessors (like ada-002). It generates 1536-dimensional vectors that capture nuanced semantic relationships. For example, in this vector space, the mathematical distance between "nitrogen dioxide" and "respiratory inflammation" is minimized, allowing the system to retrieve relevant health warnings even if the user's query uses different terminology (e.g., "breathing smog").[8]

2.4 Integration via LangChain

To orchestrate this pipeline, we utilize the LangChain framework. LangChain provides a standardized interface for Document Loaders and Vector Stores. Its SupabaseVectorStore class abstracts the complexity of the SQL INSERT statements. It automatically handles the serialization of the vector data and the JSON metadata, batching the upserts to respect database connection limits. This abstraction allows the data engineering team to focus on the logic of chunking rather than the boilerplate of database connectivity.[8]

2.5 Developer Prompt 2: Knowledge Ingestion

This prompt directs the creation of the Python ETL script. It focuses on the specific libraries required for robust PDF parsing and the precise configuration of the splitter to respect the structure of legal documents.

Developer Prompt 2: The Knowledge Ingestion (Python RAG Script)

Context:
You are a Data Engineer building the ETL pipeline for Eco-Neuro Sentinel. The goal is to ingest the "EU Air Quality Directive" (PDF format) into the Supabase knowledge_base table to enable Retrieval Augmented Generation (RAG).
Goal:
Create a robust Python script (ingest_knowledge.py) that parses the PDF, chunks the text intelligently, generates embeddings using OpenAI, and upserts the data to Supabase.
Dependencies:

- langchain
- langchain-community
- langchain-openai [8]
- supabase (official Python client)
- pypdf (for PDF extraction)

- python-dotenv (for managing API keys)

Logic & Flow:

1. **Configuration:**
   - Load SUPABASE_URL, SUPABASE_SERVICE_KEY, and OPENAI_API_KEY from a .env file.
   - **Security Note:** Use the SERVICE_KEY (admin) because the RLS policy on knowledge_base likely restricts INSERT access to admins only.
2. **PDF Loading:**
   - Use PyPDFLoader from langchain_community.document_loaders to load EU_Air_Quality_Directive.pdf.
   - Implement a cleaning step to strip excessive newlines or artifacts (e.g., replace \n with spaces where appropriate to maintain sentence flow).
3. **Semantic Chunking:**
   - Instantiate RecursiveCharacterTextSplitter.
   - **Settings:** chunk_size=1000, chunk_overlap=200.
   - **Rationale:** Legal directives have dense clauses; 1000 characters usually capture a full article or subsection, and the overlap ensures that context (e.g., "Article 13 states...") isn't lost at the cut.
4. **Embedding & Storage:**
   - Initialize OpenAIEmbeddings with model="text-embedding-3-small".[8]
   - Initialize SupabaseVectorStore utilizing the supabase-js client (via the python wrapper).
   - Target the knowledge_base table.
   - Target the match_documents query name (for validation).
   - Use the from_documents class method to perform the embedding generation and upsert in batches.
5. **Error Handling:**
   - Wrap the upsert logic in a try/except block to catch connection timeouts or API rate limits. Print the number of chunks successfully added.

Output:
Provide the complete Python script code, including the necessary pip install commands in a comment block.

Phase III: The Neuro-Symbolic Agent (Supabase Edge Function)

3.1 Architectural Theory: Serverless Intelligence at the Edge

The "Neuro-Symbolic Agent" is the cognitive core of the system. It is deployed as a Supabase

Edge Function, running on the Deno runtime. This architecture offers two critical advantages:

1. **Latency:** Edge Functions are distributed globally. The code executes in a data center geographically close to the user, minimizing the round-trip time for the API request.[9]
2. **Security Sandbox:** Deno is secure by default. It requires explicit permissions to access the network or environment variables, reducing the attack surface.

The "Neuro-Symbolic" designation refers to the agent's workflow. It combines **Symbolic AI** (deterministic steps: PII redaction via Regex, database lookups via SQL, spatial queries via PostGIS) with **Neural AI** (probabilistic reasoning via GPT-4o). This hybrid approach mitigates the risk of "hallucinations" common in pure LLM applications by grounding the model in retrieved facts (RAG) and validated data (Sensor readings).

3.2 Privacy Engineering: PII Redaction

Handling medical data requires strict adherence to privacy standards. Before the patient's symptom description is sent to OpenAI's servers for analysis, it must be scrubbed of Personally Identifiable Information (PII). The user specifically requested redact-pii. While the Deno ecosystem is distinct from Node.js, Supabase Edge Functions now support NPM modules natively using the npm: specifier.[1] This allows us to import robust PII redaction libraries.

However, recognizing that specific libraries may have varying compatibility, the architectural pattern enforces a "Defense in Depth" strategy. We prioritize the requested library but structure the code to allow for fallback regex patterns for critical identifiers (Email, Phone, SSN). This ensures that even if the library fails to identify a specific edge case, the most sensitive vectors are covered. The redacted text is what generates the embedding and what is sent to the LLM, ensuring the third-party AI provider never sees the raw PII.[11]

3.3 The Triage Workflow

1. **Ingest:** Receive JSON payload { symptoms, coordinates }.
2. **Sanitize:** Apply redact-pii to the symptoms string.
3. **Retrieve (Symbolic):**
   - Generate embedding for *redacted* symptoms.
   - Execute match_documents RPC to get regulatory context.
   - Execute ST_Contains spatial query to get air_quality for the user's sector.
4. **Reason (Neural):** Construct a prompt for GPT-4o that includes the redacted symptoms, the retrieved regulation, and the pollution level. The system prompt explicitly restricts the AI to a "Triage" role, forbidding definitive medical diagnoses.
5. **Respond:** Return the risk assessment and the redacted version of the text (for user verification).

3.4 Developer Prompt 3: The Neuro-Symbolic Agent

This prompt guides the creation of the Edge Function, focusing on the integration of NPM modules in Deno and the strict sequencing of privacy and logic steps.

Developer Prompt 3: The Neuro-Symbolic Agent (Supabase Edge Function)

Context:
You are a Backend Developer implementing the core logic of Eco-Neuro Sentinel on Supabase Edge Functions (Deno). This function must act as a secure proxy between the user and OpenAI, enforcing privacy and providing context-aware medical triage.
Goal:
Create a TypeScript Edge Function (analyze-risk) that sanitizes input, performs RAG, fetches environmental data, and queries GPT-4o.
Dependencies:

- npm:openai [10]
- npm:@supabase/supabase-js
- npm:redact-pii (primary redaction tool) OR a suitable alternative like npm:google-libphonenumber combined with custom Regex if redact-pii proves incompatible in the Deno environment. *For this prompt, attempt to use redact-pii via NPM import*.
- Deno native APIs (Deno.env, Deno.serve).

Logic & Flow:

1. **Setup & Auth:**
   - Import createClient from @supabase/supabase-js.
   - Import OpenAI from openai.
   - Initialize clients using SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY (needed to select from patient_logs if logging is required, and knowledge_base), and OPENAI_API_KEY.
2. **Request Handler:**
   - Use Deno.serve to listen for POST requests.
   - Parse the body: { symptom_description: string, user_lat: number, user_long: number }.
3. **Step 1: Privacy Firewall (PII Redaction):**
   - Import Redactor from npm:redact-pii.
   - Configure the redactor to target emails, phone numbers, and names.
   - Process symptom_description -> redacted_symptoms.
   - *Fallback:* Include a comment or a backup Regex function for emails/phones to ensure redundancy.

4. **Step 2: Context Retrieval (RAG):**
    - Call OpenAI embeddings.create with input: redacted_symptoms and model: text-embedding-3-small.
    - Call Supabase RPC match_documents with the resulting vector. Retrieve the top 3 chunks of regulatory text.
5. **Step 3: Environmental Context:**
    - Perform a database query on air_quality.
    - Use PostGIS logic: ST_Contains(boundary, ST_SetSRID(ST_MakePoint(user_long, user_lat), 4326)).
    - Fetch the pm25 and pm10 values for that sector.
6. **Step 4: Neuro-Symbolic Inference:**
    - Call chat.completions.create (GPT-4o).
    - **System Prompt:** "You are an expert medical triage assistant using EU Air Quality regulations. You must analyze the patient's symptoms in the context of the provided pollution levels and regulations. Output a JSON object with keys: 'risk_level' (Low/Medium/High), 'advisory' (Actionable advice), and 'relevant_regulation' (Citation from context)."
    - **User Prompt:** "Pollution: [PM2.5 values]. Regulation Context:. Symptoms:."
7. **Response:**
    - Return the GPT-4o JSON response to the client.

Output:
Provide the complete index.ts file.

Phase IV: The Frontend Dashboard (Next.js + Leaflet)

4.1 Architectural Theory: The SSR/CSR Dichotomy in Mapping

The frontend architecture utilizes Next.js 14, enabling a hybrid of Server-Side Rendering (SSR) for performance/SEO and Client-Side Rendering (CSR) for interactivity. However, geospatial libraries like Leaflet pose a specific challenge in this environment. Leaflet is designed to manipulate the DOM directly and relies on the global window object, which is absent during the server-side build process. Attempting to import react-leaflet in a standard Next.js page results in a "Window is not defined" crash.[13]

To resolve this, we employ **Dynamic Imports with SSR Disabling**. Next.js provides the next/dynamic utility, which allows us to import a component asynchronously. By setting the { ssr: false } option, we instruct the Next.js compiler to exclude the map component from the server bundle entirely. It is only hydrated once the JavaScript reaches the client browser. This architectural pattern is essential for integrating legacy DOM-based libraries into modern React Server Component (RSC) architectures.[13]

4.2 Real-Time State Synchronization via WebSockets

A static map is insufficient for an environmental monitoring system; pollution is a dynamic variable. We leverage Supabase Realtime, which hooks into PostgreSQL's replication log (WAL). When a record in the air_quality table is updated (e.g., by an IoT sensor ingestion script), Postgres emits a signal. Supabase translates this into a WebSocket message broadcast to all subscribed clients.

In the React application, this is handled via the useEffect hook. The client subscribes to the postgres_changes channel for the air_quality table. Upon receiving an UPDATE event, the application state is mutated, triggering a re-render of the Map polygons. This ensures that if a pollution spike occurs in "Sector 1," the polygon on the user's dashboard turns red instantly, without requiring a page refresh. This responsiveness is critical for the "Sentinel" aspect of the application—alerting users to danger as it happens.

4.3 Developer Prompt 4: The Frontend Dashboard

This prompt guides the construction of the UI, explicitly handling the complex integration of Leaflet within Next.js and the setup of the Realtime subscription listeners.

Developer Prompt 4: The Frontend Dashboard (Next.js + Leaflet)

Context:
You are a Frontend Architect using Next.js 14 (App Router). You are building the main interface for Eco-Neuro Sentinel, which features a live pollution map of Bucharest and a symptom logging form.
Goal:
Create the page structure and components, ensuring compatibility between Leaflet and Next.js SSR, and enabling real-time data updates.
Dependencies:

- react-leaflet, leaflet, leaflet-defaulticon-compatibility (fixes missing marker icons).[15]
- @supabase/supabase-js
- next/dynamic

Components & Logic:

1. **MapComponent.tsx (Client Component):**
    - **Directives:** Use 'use client'.
    - **Imports:** MapContainer, TileLayer, Polygon, Popup from react-leaflet.
    - **Props:** pollutionData (Array of sector objects).
    - **Logic:**

        - Render a MapContainer centered on Bucharest (44.4268, 26.1025).
        - Iterate through pollutionData. Render a Polygon for each sector.
        - **Styling:** Dynamic fillColor based on PM2.5 levels:
            - < 10: Green (#4ade80)
            - 10 - 25: Orange (#fb923c)
            - 25: Red (#ef4444)
        - **Popup:** Display the Sector Name and PM2.5 value on click.
2. **page.tsx (Main Page):**
    - **Dynamic Import:** Import MapComponent using next/dynamic with { ssr: false } to prevent "window is not defined" errors. Provide a loading fallback (e.g., a skeleton div).
    - **State Management:** Use useState to hold pollutionData.
    - **Real-Time Subscription:**
        - In useEffect, initialize supabase.channel.
        - Subscribe to postgres_changes on event: 'UPDATE', table: 'air_quality'.
        - On receiving a payload, update the specific sector in the pollutionData state array to reflect the new values immediately.
3. **Symptom Logger Form:**
    - Create a simple form component (textarea for symptoms).
    - onSubmit: Make a POST request to the Supabase Edge Function analyze-risk (endpoint URL from environment variables).
    - Handle the response: Display the "Risk Level" and "Advice" in a styled Alert box.

Output:
Provide the code for components/MapComponent.tsx and app/page.tsx, clearly commenting the dynamic import strategy and the subscription logic.

Phase V: The Synthetic Validator (Python ML)

5.1 Architectural Theory: Synthetic Data for Ethical Validation

In the development of medical AI systems, access to real patient data is often bottlenecked by ethical approvals and privacy constraints. To validate the Eco-Neuro Sentinel's logic before clinical pilots, we utilize **Synthetic Data Generation**. This involves creating fictitious patient profiles that mirror the statistical distribution of real populations without corresponding to actual individuals.

We use the Faker library, configured with the ro_RO (Romanian) locale, to generate culturally plausible names and addresses. More importantly, we programmatically generate symptom correlations. For instance, the script injects a causal relationship: "If PM2.5 > 30, then 60% of generated logs should report 'dyspnea' or 'wheezing'." This synthetic dataset allows us to verify if the Neuro-Symbolic Agent correctly identifies high-risk scenarios.

5.2 Predictive Modeling at the Edge

Beyond validation, this data drives the "Sentinel" predictive capability. We train a lightweight Machine Learning model—specifically a **Random Forest Regressor** via scikit-learn—to forecast symptom severity based on environmental inputs. Unlike Deep Learning models which are computationally heavy, a Random Forest is efficient enough to be serialized (pickled) and potentially run within a microservice or a larger container.

The model takes pm25 and pm10 as independent variables (features) and predicts the severity_score (target). By training on the synthetic data (which encodes the pollution-symptom correlation), the model learns to anticipate public health risks. For example, if the weather forecast predicts a PM2.5 spike tomorrow, the model can predict a corresponding rise in emergency room triage severity.

5.3 Developer Prompt 5: The Synthetic Validator

This prompt orchestrates the generation of the test dataset and the training of the predictive model, establishing the feedback loop between environmental data and health outcomes.

Developer Prompt 5: The Synthetic Validator (Python ML)

Context:
You are a Machine Learning Engineer. You need to generate a synthetic dataset to load-test the Eco-Neuro Sentinel and train a predictive model that correlates air quality with symptom severity.
Goal:
Create a Python script (validator.py) that generates realistic mock data, inserts it into Supabase, and trains a Scikit-Learn Random Forest model.
Dependencies:

- faker (for PII generation)
- scikit-learn (for ML)
- pandas, numpy
- supabase

Logic & Flow:

1. **Synthetic Data Generator:**
    - Initialize Faker('ro_RO').
    - **Air Quality Mock:** Generate 50 records representing varying pollution levels in Bucharest sectors over the last week.
    - **Patient Log Mock:** Generate 200 patient logs.
    - **Correlation Logic:** Implement a probabilistic function:

        - If air_quality.pm25 > 25 (High), assign severity_score between 7-10 and include keywords "asthma", "choking", "cough" in symptom_text.
        - If air_quality.pm25 < 10 (Low), assign severity_score between 1-3.
    - **Upsert:** Insert these records into air_quality and patient_logs via the Supabase client.
2. **Data Preparation:**
    - Query the database to join patient_logs with air_quality (based on location/sector).
    - Load the result into a Pandas DataFrame.
    - Features (X): pm25, pm10.
    - Target (y): severity_score.
3. **Model Training:**
    - Split the data: train_test_split(X, y, test_size=0.2).
    - Initialize RandomForestRegressor(n_estimators=100).
    - Fit the model on the training set.
4. **Forecasting & Validation:**
    - Evaluate the model on the test set (calculate MSE).
    - **Simulation:** Run a prediction for a hypothetical scenario: "PM2.5 = 45". Print the predicted average symptom severity.
    - **Serialization:** Save the trained model to a file (model.pkl) for potential future deployment.

Output:
Provide the complete Python script validator.py.

Strategic Conclusion

The architectural blueprint for **Eco-Neuro Sentinel** represents a sophisticated synthesis of modern cloud-native technologies. By decoupling the persistence layer (Supabase/Postgres) from the reasoning layer (OpenAI/Edge Functions) and the presentation layer (Next.js), the system achieves strict separation of concerns, modularity, and scalability.

Key technical victories in this design include:

- **Privacy-First Architecture:** The use of RLS and Edge-side PII redaction ensures that the system is GDPR-aware by design, creating a "Zero Trust" environment for sensitive patient data.
- **High-Dimensional Recall:** The HNSW indexing strategy guarantees that as the regulatory knowledge base expands, the retrieval time remains logarithmic, ensuring real-time responsiveness.
- **Reactive User Experience:** The integration of Supabase Realtime with Leaflet maps creates a "living" dashboard that reflects the dynamic nature of environmental health without manual polling.
- **Neuro-Symbolic Reliability:** By grounding the LLM's stochastic outputs in the deterministic data of the EU Directives (via RAG), the system significantly reduces the risk

of "hallucinations," a critical requirement for any medical-adjacent application.

This report provides the roadmap; the accompanying prompts provide the vehicle. Execution of these prompts will result in a functional, secure, and intelligent application ready for pilot deployment in the Bucharest metropolitan area, capable of transforming raw environmental data into protective health insights.

Works cited

1. NPM compatibility | Supabase Features, accessed on December 6, 2025, https://supabase.com/features/npm-compatibility
2. pgvector: Embeddings and vector similarity | Supabase Docs, accessed on December 6, 2025, https://supabase.com/docs/guides/database/extensions/pgvector
3. HNSW indexes | Supabase Docs, accessed on December 6, 2025, https://supabase.com/docs/guides/ai/vector-indexes/hnsw-indexes
4. pgvector 0.6.0: 30x faster with parallel index builds - Supabase, accessed on December 6, 2025, https://supabase.com/blog/pgvector-fast-builds
5. Increase vector lookup speeds by applying an HSNW index - Supabase, accessed on December 6, 2025, https://supabase.com/docs/guides/troubleshooting/increase-vector-lookup-speeds-by-applying-an-hsnw-index-ohLHUM
6. langchain.vectorstores.supabase - Read the Docs, accessed on December 6, 2025, https://lagnchain.readthedocs.io/en/stable/_modules/langchain/vectorstores/supabase.html
7. LangChain | Supabase Docs, accessed on December 6, 2025, https://supabase.com/docs/guides/ai/langchain
8. SupabaseVectorStore - LangChain.js, accessed on December 6, 2025, https://api.js.langchain.com/classes/langchain_community_vectorstores_supabase.SupabaseVectorStore.html
9. Edge Functions | Supabase Docs, accessed on December 6, 2025, https://supabase.com/docs/guides/functions
10. Supabase Edge Functions - Deploy JavaScript globally in seconds, accessed on December 6, 2025, https://supabase.com/edge-functions
11. Redact personally identifiable information (PII) - Snowflake Documentation, accessed on December 6, 2025, https://docs.snowflake.com/en/user-guide/snowflake-cortex/redact-pii
12. Redact PII from traces - Weights & Biases Documentation, accessed on December 6, 2025, https://docs.wandb.ai/weave/guides/tracking/redact-pii
13. Guides: Lazy Loading | Next.js, accessed on December 6, 2025, https://nextjs.org/docs/pages/guides/lazy-loading
14. Nextjs with react-leaflet - SSR, webpack | window not defined, icon not found, accessed on December 6, 2025, https://stackoverflow.com/questions/77978480/nextjs-with-react-leaflet-ssr-web

pack-window-not-defined-icon-not-found
15. How to use react-leaflet in Nextjs with TypeScript (Surviving it) | by Andres M. Prieto, accessed on December 6, 2025, https://andresmpa.medium.com/how-to-use-react-leaflet-in-nextjs-with-typescript-surviving-it-21a3379d4d18